

JS8Call API, Version v2.3.2-devel

N0GQ, Jeff Francis

Overview

JS8Call has an API that allows for manipulation of program and modem state, sending and receiving data over RF, and control of the attached radio.

The API in versions of JS8Call from 2019 through 2025 was limited. Many functions did not work correctly or completely, and the program features controllable from the API was very limited. As of v2.3.2 in late 2025, the API has been fixed and greatly expanded.

The API for JS8Call differs from the traditional synchronous REST API where commands are sent and the API call blocks until the reply is received. Instead, the API works in an asynchronous manner, where commands are sent, and the API call returns immediately, without any result. JS8Call then formulates a reply and sends it back to the client once the requested task is complete. The reply contains an identifier that links it back to the original request. Replies may be sent out of order with respect to their corresponding requests, though this is rare. Replies are typically returned within a few hundred milliseconds after the originating request, though there are exceptions. API payloads and returned values are typically JSON blobs, except where noted. Most, but not all, API calls return a unique “ID” value. This is typically ignored.

This architecture requires a somewhat different program flow for API clients than the traditional “ask, wait, receive, continue” design. The typical approach is to have a multi-threaded client. The first thread reads requests from a user program queue, and sends them one at a time to JS8Call. The second thread continuously receives and processes replies from JS8Call, and links those replies back to their original request and delivers them in a queue to the user. Yes, it’s as ugly as it sounds.

There are several additional idiosyncracies that should be noted:

- Any submitted JSON should be a single ASCII string terminated with a single `.`. The pretty-printed examples in the documentation will not work as-is.
- If the result of an API call is a single value, it is often returned as the value of the JSON field “value”. But not always.
- If multiple values are returned, they are enumerated as key/value pairs inside of the `params{}` hash. In a few cases, single values are returned as part of `params{}`.
- `_ID` is almost always returned as part of the JSON hash. It is a unique integer, but it’s generally not useful.
- `ID` is not the same as `_ID`.

While it’s certainly viable, and sometimes necessary, to talk to the API directly, in many cases, it’s easier to use a library that abstracts the complexity from the programmer, allowing focus on the problem at hand rather than the complexities of the API. One such library, written by the author of this document for python3 clients, is `js8net`. It is generally kept in sync with the current release of `JS8Call`. Using a library such as this makes programming considerably easier, as it allows for simple function calls and replies, rather than managing threads and queues of sent and received data. The code and documentation can be found here:

<https://github.com/jfrancis42/js8net>

Note that access to the API is disabled by default in a new `JS8Call` installation. It must be enabled in the setup window. There are both TCP and UDP versions of the API available, though it’s traditional to use the TCP endpoints. The default TCP port is 2442. It’s also important to note that by default, the API only listens on the loopback interface, meaning it only listens to API clients running on the same machine as `JS8Call`. If it is desired to control `JS8Call` from machines other than this machine, you must bind to a non-loopback address. This is typically done by using `0.0.0.0`. Note that there is substantial risk in exposing the `JS8Call` API to the Internet.

The various API endpoints are listed below. The “Endpoint” field is the name of the API endpoint to be called. The “Return Designator” is the identifier included in the reply from the API (not all calls have replies). “Parameters” lists the required information (if any) to be sent to the API endpoint. “Return Values” describes the data (if any) returned from the API call. Each API endpoint description includes example sent and received data, where relevant.

Note that calling an non-existent API endpoint will result in no action and no error. API endpoints that do not send data are idempotent. If they specify a state that already exists, for example, enabling Spots, no action occurs.

SYSTEM Endpoints

SYSTEM endpoints return information relating the the system that you are connected to. For example, the software version of JS8Call, the operating system, and the user information.

Endpoint: SYSTEM.JS8_VERSION

Description

Return the JS8Call version info.

Calling Value

None

Calling Params

None

Return Value

version string

Return Params

- _ID - Unique transaction ID

Return Type

SYSTEM.JS8_VERSION

Example Sent Data

```
{
  "params": {},
  "type": "SYSTEM.JS8_VERSION",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261664936179
  },
  "type": "SYSTEM.JS8_VERSION",
  "value": "2.3.2-devel"
}
```

Notes

This endpoint returns the current running version of JS8Call. This can be used by API clients to know which API endpoints are valid for this version of code, and to know what return data to expect. The API will likely evolve over time, and this allows API users to craft software that takes advantage of new JS8Call features while maintaining backwards compatibility with older versions. Note that this call did not exist in versions of JS8Call from 2019 until late 2025, so making this call and receiving no reply implies an older version of JS8Call. The API was unchanged from 2019 until late 2025.

Endpoint: SYSTEM.OS_INFO

Description

Returns user and operating system information.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- HOMEDIR - The path for the home directory of the user running JS8Call.
- OS_KERNEL - The operating system.
- OS_KERNEL_VERSION - What version of the OS.
- OS_NAME - The variant of the OS.
- USERNAME - The username of the user running JS8Call.
- _ID - Unique transaction ID

Return Type

SYSTEM.OS_INFO

Example Sent Data

```
{
  "params": {},
  "type": "SYSTEM.OS_INFO",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "HOMEDIR": "/home/jfrancis",
    "OS_KERNEL": "linux",
    "OS_KERNEL_VERSION": "6.17.2-arch1-1",
    "OS_NAME": "Arch Linux",
    "USERNAME": "jfrancis",
    "_ID": 261665182423
  },
  "type": "SYSTEM.OS_INFO",
  "value": ""
}
```

Notes

This endpoint did not exist in versions of JS8Call from 2019 through late 2025. Calling this API in older versions will result in no data returned.

WINDOW Endpoints

WINDOW endpoints relate to the display of the JS8Call application.

Endpoint: WINDOW.RAISE

Description

Raises the JS8Call window to the “top” on the screen.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

None

Return Type

None

Example Sent Data

```
{  
  "params": {},  
  "type": "WINDOW.RAISE",  
  "value": ""  
}
```

Example Reply

None

Notes

This is the only API call that returns nothing whatsoever. This function call does not appear to work with most Linux window managers. ToDo: fix the API to return something.

STATION Endpoints

STATION endpoints relate to the configuration of the JS8Call software itself.

Endpoint: STATION.SPOT_ON

Description

Assuming spotting is configured in the setup, this will turn the SPOT button on and enable spots to be sent via the Internet.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

STATION.SPOT

Example Sent Data

```
{
  "params": {},
  "type": "STATION.SPOT_ON",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261673311050
  },
  "type": "STATION.SPOT",
  "value": ""
}
```

Notes

None

Endpoint: STATION.SPOT_OFF

Description

Assuming spotting is configured in the setup, this will turn the SPOT button off and disable spots being sent via the Internet.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

STATION.SPOT

Example Sent Data

```
{
  "params": {},
  "type": "STATION.SPOT_OFF",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261673365782
  },
  "type": "STATION.SPOT",
  "value": ""
}
```

Notes

None

Endpoint: STATION.AUTOREPLY_ON

Description

Turn on Autoreply.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

STATION.AUTOREPLY

Example Sent Data

```
{
  "params": {},
  "type": "STATION.AUTOREPLY_ON",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261673762040
  },
  "type": "STATION.AUTOREPLY",
  "value": ""
}
```

Notes

None

Endpoint: STATION.AUTOREPLY_OFF

Description

Turn off Autoreply.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

STATION.AUTOREPLY

Example Sent Data

```
{
  "params": {},
  "type": "STATION.AUTOREPLY_OFF",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261673725765
  },
  "type": "STATION.AUTOREPLY",
  "value": ""
}
```

Notes

None

Endpoint: STATION.HB_ON

Description

Enables heartbeat networking.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

STATION.HB

Example Sent Data

```
{
  "params": {},
  "type": "STATION.HB_ON",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261673856598
  },
  "type": "STATION.HB",
  "value": ""
}
```

Notes

None

Endpoint: STATION.HB_OFF

Description

Disables heartbeat networking.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

STATION.HB

Example Sent Data

```
{
  "params": {},
  "type": "STATION.HB_OFF",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261673826963
  },
  "type": "STATION.HB",
  "value": ""
}
```

Notes

None

Endpoint: STATION.HB_ACKS_ON

Description

Enable Automatic acknowledgement of Heartbeats.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

STATION.HB_ACKS

Example Sent Data

```
{
  "params": {},
  "type": "STATION.HB_ACKS_ON",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261674038225
  },
  "type": "STATION.HB_ACKS",
  "value": ""
}
```

Notes

None

Endpoint: STATION.HB_ACKS_OFF

Description

Disable automatic acknowledgement of heartbeats.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

STATION.HB_ACKS

Example Sent Data

```
{
  "params": {},
  "type": "STATION.HB_ACKS_OFF",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261674003190
  },
  "type": "STATION.HB_ACKS",
  "value": ""
}
```

Notes

None

Endpoint: STATION.GET_CALLSIGN

Description

Get my callsign.

Calling Value

None

Calling Params

None

Return Value

The configured callsign.

Return Params

- _ID - Unique transaction ID

Return Type

STATION.CALLSIGN

Example Sent Data

```
{
  "params": {},
  "type": "STATION.GET_CALLSIGN",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261674037224
  },
  "type": "STATION.CALLSIGN",
  "value": "N0GQ"
}
```

Notes

None

Endpoint: STATION.GET_GRID

Description

Get my grid locator.

Calling Value

None

Calling Params

None

Return Value

The configured grid square.

Return Params

- `_ID` - Unique transaction ID

Return Type

STATION.GRID

Example Sent Data

```
{
  "params": {},
  "type": "STATION.GET_GRID",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261674101254
  },
  "type": "STATION.GRID",
  "value": "DM79PI95FH"
}
```

Notes

None

Endpoint: STATION.SET_GRID

Description

Set my grid locator.

Calling Value

Desired grid locator.

Calling Params

None

Return Value

The grid square that was just set.

Return Params

- _ID - Unique transaction ID

Return Type

STATION.GRID

Example Sent Data

```
{
  "params": {},
  "type": "STATION.SET_GRID",
  "value": "DM79"
}
```

Example Reply

```
{
  "params": {
    "_ID": 261675669492
  },
  "type": "STATION.GRID",
  "value": "DM79"
}
```

Notes

None

Endpoint: STATION.GET_INFO

Description

Get my station info field.

Calling Value

None

Calling Params

None

Return Value

The value of the Info field.

Return Params

- _ID - Unique transaction ID

Return Type

STATION.INFO

Example Sent Data

```
{
  "params": {},
  "type": "STATION.GET_INFO",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261675867099
  },
  "type": "STATION.INFO",
  "value": "SunSDR2 Pro + amp, 15W, OCFD AT 35FT"
}
```

Notes

None

Endpoint: STATION.SET_INFO

Description

Set my station info field.

Calling Value

The desired contents of the Info field.

Calling Params

The desired value for the Info field.

Return Value

The new Info field value.

Return Params

- `_ID` - Unique transaction ID

Return Type

STATION.INFO

Example Sent Data

```
{
  "params": {},
  "type": "STATION.SET_INFO",
  "value": "SunSDR2 Pro + amp, 15W, OCFD AT 35FT"
}
```

Example Reply

```
{
  "params": {
    "_ID": 261676369100
  },
  "type": "STATION.INFO",
  "value": "SunSDR2 Pro + amp, 15W, OCFD AT 35FT"
}
```

Notes

Notes: Changes the data structure in memory and uses the correct new value going forward. Does not persist to disk and the changes do not show up in the GUI. Works fine for API only usage.

Endpoint: STATION.GET_STATUS

Description

Get my station status field.

Calling Value

None

Calling Params

None

Return Value

The value of the Status field.

Return Params

- `_ID` - Unique transaction ID

Return Type

STATION.STATUS

Example Sent Data

```
{
  "params": {},
  "type": "STATION.GET_STATUS",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261676503005
  },
  "type": "STATION.STATUS",
  "value": "Groovy, baby."
}
```

Notes

None

Endpoint: STATION.SET_STATUS

Description

Set my station status field.

Calling Value

Desired status message.

Calling Params

None

Return Value

The new Status field value.

Return Params

- `_ID` - Unique transaction ID

Return Type

STATION.STATUS

Example Sent Data

```
{
  "params": {},
  "type": "STATION.SET_STATUS",
  "value": "Groovy, baby."
}
```

Example Reply

```
{
  "params": {
    "_ID": 261676502755
  },
  "type": "STATION.STATUS",
  "value": "groovy, baby"
}
```

Notes

Notes: Changes the data structure in memory and uses the correct new value going forward. Does not persist to disk and the changes do not show up in the GUI. Works fine for API only usage.

MODE Endpoints

MODE endpoints relate to the configuration of the JS8Call modem (speed, etc).

Endpoint: MODE.GET_SPEED

Description

Return the current modem speed (as an integer).

Calling Value

None

Calling Params

None

Return Value

An integer value representing the current modem speed.

Return Params

- SPEED - An integer representing the current modem speed.
- _ID - Unique transaction ID

Return Type

MODE.SPEED

Example Sent Data

```
{
  "params": {},
  "type": "MODE.GET_SPEED",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "SPEED": 0,
    "_ID": 261676646148
  },
  "type": "MODE.SPEED",
```

```
"value": ""  
}
```

Notes

slow==4, normal==0, fast==1, turbo==2

Endpoint: MODE.SET_SPEED

Description

Set the modem speed.

Calling Value

None

Calling Params

- SPEED - An integer representing the desired speed.

Return Value

The new modem speed.

Return Params

- SPEED - An integer representing the new speed that was set.
- _ID - Unique transaction ID

Return Type

SPEED

Example Sent Data

```
{  
  "params": {  
    "SPEED": 0  
  },  
  "type": "MODE.SET_SPEED",  
  "value": ""  
}
```

Example Reply

```
{  
  "params": {
```

```
    "SPEED": 0,  
    "_ID": 261676969756  
  },  
  "type": "MODE.SPEED",  
  "value": ""  
}
```

Notes

slow==4, normal==0, fast==1, turbo==2

INBOX Endpoints

INBOX endpoints allow reading and manipulation of the user's Inbox.

Endpoint: INBOX.GET_MESSAGES

Description

Return all messages in the inbox.

Calling Value

None

Calling Params

- CALLSIGN - (optional) Call sign to filter on.

Return Value

None

Return Params

- MESSAGES[] - A JSON array containing each message.
- _ID - Unique transaction ID

MESSAGES[] contains:

- params{} - contains message contents
- type - contains READ or UNREAD, indicating status of this message
- value - always contains a null string ("")

params{} contains:

- CMD - Always contains " MSG "

- DIAL - The radio dial freq in hz where the message was heard.
- FREQ - The actual carrier frequency where the message was heard.
- FROM - The callsign that originated the message.
- OFFSET - The carrier offset within the passband in hz.
- PATH - A list of callsigns that transferred the message.
- SNR - The SNR of the signal.
- SUBMODE - An integer representing the speed of the transmitting modem.
- TDRIFT - Seconds of drift between RX and TX station.
- TEXT - The contents of the message.
- TO - The callsign the message is intended for.
- UTC - UTC Timestamp when the message was received.
- _ID - Unique transaction ID for the individual message.

Return Type

INBOX.MESSAGES

Example Sent Data

```
{
  "params": {},
  "type": "INBOX.GET_MESSAGES",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "MESSAGES": [
      {
        "params": {
          "CMD": " MSG ",
          "DIAL": 7078000,
          "FREQ": 7080048,
          "FROM": "N0CLU",
          "OFFSET": 2048,
          "PATH": "N0CLU",
          "SNR": -9,
          "SUBMODE": 0,
          "TDRIFT": 0.13999998569488525,
          "TEXT": "TESTING 1 2 3... 73",
          "TO": "N0GQ",
          "UTC": "2025-10-20 01:31:56",
          "_ID": "261624763058"
        }
      }
    ]
  }
}
```

```
    },
    "type": "READ",
    "value": ""
  }
],
"_ID": 261677015884
},
"type": "INBOX.MESSAGES",
"value": ""
}
```

Notes

None

Endpoint: INBOX.STORE_MESSAGE

Description

Store a message locally for another user to pick up.

Calling Value

None

Calling Params

- CALLSIGN - The callsign for whom the message is intended.
- TEXT - The contents of the message to be stored.

Return Value

None

Return Params

- ID - The unique message ID for this message, as displayed to the recipient.
- _ID - Unique transaction ID

Return Type

INBOX.MESSAGE

Example Sent Data

```
{
  "params": {
```

```
    "CALLSIGN": "NØCLU",
    "TEXT": "NOW IS THE TIME FOR ALL GOOD MEN..."
  },
  "type": "INBOX.STORE_MESSAGE",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "ID": 2,
    "_ID": 261677155107
  },
  "type": "INBOX.MESSAGE",
  "value": ""
}
```

Notes

None

RIG Endpoints

RIG endpoints relate to the configuration and state of the attached transceiver.

Endpoint: RIG.TUNE_ON

Description

Start tuning tone.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

RIG.TUNE

Example Sent Data

```
{
  "params": {},
  "type": "RIG.TUNE_ON",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261677299410
  },
  "type": "RIG.TUNE",
  "value": ""
}
```

Notes

This keys the transmitter and sends a CW tone to enable the user to tune his antenna (or triggers the autotuner, as appropriate to the rig).

Endpoint: RIG.TUNE_OFF

Description

Stop tuning tone.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

This is the return designator.

Example Sent Data

```
{
  "params": {},
  "type": "RIG.TUNE_OFF",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261677300510
  },
  "type": "RIG.TUNE",
  "value": ""
}
```

Notes

None

Endpoint: RIG.GET_FREQ

Description

Get the current radio Frequency

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- **FREQ** - The transmit carrier frequency in hz.
- **DIAL** - The current radio dial frequency in hz.
- **OFFSET** - The offset of the actual signal within the passband in hz.
- **_ID** - Unique transaction ID

Return Type

RIG.FREQ

Example Sent Data

```
{
  "params": {},
  "type": "RIG.GET_FREQ",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "DIAL": 18104000,
    "FREQ": 18105984,
    "OFFSET": 1984,
    "_ID": 261677481069
  },
  "type": "RIG.FREQ",
  "value": ""
}
```

Notes

None

Endpoint: RIG.SET_FREQ

Description

Set the current Frequency

Calling Value

None

Calling Params

- DIAL - The desired radio dial frequency in hz.
- OFFSET - The desired offset of the actual signal within the passband in hz.

Return Value

None

Return Params

- DIAL - The radio dial frequency in hz.
- OFFSET - The offset of the actual signal within the passband in hz.
- FREQ - The carrier frequency in hz.
- SELECTED - Unknown - ToDo: Figure this out.
- SPEED - An integer representing the current modem speed.
- _ID - Unique transaction ID

Return Type

STATION.STATUS

Example Sent Data

```
{
  "params": {
    "DIAL": 18104000,
    "OFFSET": 1984
  },
  "type": "RIG.SET_FREQ",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "DIAL": 18104000,
    "FREQ": 18105984,
    "OFFSET": 1984,
    "SELECTED": "",
    "SPEED": 0,
    "_ID": "261677480819"
  },
  "type": "STATION.STATUS",
}
```

```
"value": ""  
}
```

Notes

None

RX Endpoints

RX endpoints relate to data heard over the air.

Endpoint: RX.MULTI_ON

Description

Enable simultaneous decoding of all speeds.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

RX.MULTI

Example Sent Data

```
{  
  "params": {},  
  "type": "RX.MULTI_ON",  
  "value": ""  
}
```

Example Reply


```
{
  "params": {
    "_ID": 261677672658
  },
  "type": "RX.MULTI",
  "value": ""
}
```

Notes

None

Endpoint: RX.MULTI_OFF

Description

Disable simultaneous decoding of all speeds.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

RX.MULTI

Example Sent Data

```
{
  "params": {},
  "type": "RX.MULTI_OFF",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261677601315
  },
  "type": "RX.MULTI",
  "value": ""
}
```

Notes

None

Endpoint: RX.RX_ON

Description

Listen to incoming signals from the radio.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- DIAL - The radio dial frequency in hz.
- OFFSET - The offset of the actual signal within the passband in hz.
- FREQ - The carrier frequency in hz.
- SELECTED - Unknown - ToDo: Figure this out.
- SPEED - An integer representing the current modem speed.
- _ID - Unique transaction ID

Return Type

STATION.STATUS

Example Sent Data

```
{
  "params": {},
  "type": "RX.RX_ON",
}
```

```
"value": ""  
}
```

Example Reply

```
{  
  "params": {  
    "DIAL": 18104000,  
    "FREQ": 18105984,  
    "OFFSET": 1984,  
    "SELECTED": "",  
    "SPEED": 0,  
    "_ID": "261677739295"  
  },  
  "type": "STATION.STATUS",  
  "value": ""  
}
```

Notes

None

Endpoint: RX.RX_OFF

Description

Do not listen to incoming signals from the radio.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

RX.RX

Example Sent Data

```
{
  "params": {},
  "type": "RX.RX_OFF",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261677715618
  },
  "type": "RX.RX",
  "value": ""
}
```

Notes

None

Endpoint: RX.GET_CALL_ACTIVITY

Description

Return the contents of the right pane.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- params{} - Contains a list of heard calls, each of which contains contact metadata.
- _ID - Unique transaction ID

params{} contains:

- a list of call signs

Each call sign hash contains:

- GRID - The grid square of the station.
- SNR - The SNR of the received signal from the station.
- UTC - A unix time_t epoch timestamp integer of the time the station was last heard.

Return Type

RX.CALL_ACTIVITY

Example Sent Data

```
{
  "params": {},
  "type": "RX.GET_CALL_ACTIVITY",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "VA7RBP": {
      "GRID": "",
      "SNR": -3,
      "UTC": 1760977032245
    },
    "VE6JXN": {
      "GRID": "",
      "SNR": -1,
      "UTC": 1760977032233
    },
    "W5SDR": {
      "GRID": "DM72DV",
      "SNR": 11,
      "UTC": 1760977032254
    },
    "_ID": 261677847034
  },
  "type": "RX.CALL_ACTIVITY",
  "value": ""
}
```

Notes

None

Endpoint: RX.GET_CALL_SELECTED

Description

Return the currently-selected call (or False, if none).

Calling Value

None

Calling Params

None

Return Value

The currently selected call in the GUI, or False if none.

Return Params

- `_ID` - Unique transaction ID

Return Type

RX.CALL_SELECTED

Example Sent Data

```
{
  "params": {},
  "type": "RX.GET_CALL_SELECTED",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261677989850
  },
  "type": "RX.CALL_SELECTED",
  "value": "W5SDR"
}
```

Notes

None

Endpoint: RX.GET_BAND_ACTIVITY

Description

Return the contents of the left pane.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- params{} - A list of ASCII-encoded integers representing the carrier offset in hz from the dial freq for ongoing messages.
- _ID - Unique transaction ID

For each offset, the data includes:

- DIAL - The radio dial frequency in hz.
- OFFSET - The offset of the actual signal within the passband in hz.
- FREQ - The carrier frequency in hz.
- SNR - The SNR of the signal.
- TEXT - The accumulated text received.
- UTC - UTC Timestamp when the last info was received.

Return Type

RX.BAND_ACTIVITY

Example Sent Data

```
{
  "params": {},
  "type": "RX.GET_BAND_ACTIVITY",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "1251": {
      "DIAL": 14078000,
      "FREQ": 14079251,
      "OFFSET": 1251,
      "SNR": -1,
      "TEXT": "VE6JXN: N0GQ HEARTBEAT SNR +05 ",
      "UTC": 1760977032233
    },
    "1772": {
      "DIAL": 14078000,
      "FREQ": 14079772,
      "OFFSET": 1772,
      "SNR": 9,
      "TEXT": "DM72DV",
      "UTC": 1760972066676
    },
    "596": {
      "DIAL": 14078000,
      "FREQ": 14078596,
      "OFFSET": 596,
      "SNR": 11,
      "TEXT": "W5SDR: N0GQ HEARTBEAT SNR +14 ",
      "UTC": 1760977032254
    },
    "656": {
      "DIAL": 14078000,
      "FREQ": 14078656,
      "OFFSET": 656,
      "SNR": -16,
      "TEXT": "KQ4TSL: @HB HEARTBEAT EM73 ",
      "UTC": 1760972126978
    },
    "703": {
      "DIAL": 14078000,
      "FREQ": 14078703,
      "OFFSET": 703,
      "SNR": -22,
      "TEXT": "KA2YNT: N0GQ HEARTBEAT SNR -11 ",
      "UTC": 1760972006808
    }
  },
}
```



```
"847": {
  "DIAL": 14078000,
  "FREQ": 14078847,
  "OFFSET": 847,
  "SNR": 11,
  "TEXT": "W5SDR: N0GQ HEARTBEAT SNR +16 ",
  "UTC": 1760972006798
},
"952": {
  "DIAL": 14078000,
  "FREQ": 14078952,
  "OFFSET": 952,
  "SNR": -3,
  "TEXT": "VA7RBP: N0GQ HEARTBEAT SNR +07 ",
  "UTC": 1760977032245
},
"_ID": 261677889126
},
"type": "RX.BAND_ACTIVITY",
"value": ""
}
```

Notes

None

Endpoint: RX.GET_TEXT

Description

Return the contents of the yellow pane.

Calling Value

None

Calling Params

None

Return Value

A single string containing all lines of text in the yellow pane, each line separated by characters.

Return Params

- `_ID` - Unique transaction ID

Return Type

RX.TEXT

Example Sent Data

```
{
  "params": {},
  "type": "RX.GET_TEXT",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261678059476
  },
  "type": "RX.TEXT",
  "value": "\n14:52:51 - (550) - N0GQ: @HB HEARTBEAT DM79 \u2662\n\n14:53:26 - (847) - W5SDR: N0GQ HEARTBEAT SNR +16 \u2662\n\n14:53:26 - (703) - KA2YNT: N0GQ HEARTBEAT SNR -11 \u2662\n\n14:53:40 - (1984) - N0GQ: W5SDR GRID? \u2662\n\n14:54:11 - (1772) - W5SDR: N0GQ GRID DM72DV \u2662\n\n14:55:27 - (950) - N0GQ: KQ4TSL HEARTBEAT SNR -16 \u2662\n\n16:16:36 - (700) - N0GQ: @HB HEARTBEAT DM79 \u2662\n\n16:17:12 - (1251) - VE6JXN: N0GQ HEARTBEAT SNR +05 \u2662\n\n16:17:12 - (952) - VA7RBP: N0GQ HEARTBEAT SNR +07 \u2662\n\n16:17:12 - (596) - W5SDR: N0GQ HEARTBEAT SNR +14 \u2662 "
}
```

Notes

None

TX Endpoints

TX endpoints relate to transmitting data.

Endpoint: TX.GET_QUEUE_DEPTH

Description

Return the number of items in the transmit queue.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- DEPTH - The current queue depth.
- _ID - Unique transaction ID

Return Type

TX.QUEUE_DEPTH

Example Sent Data

```
{
  "params": {},
  "type": "TX.GET_QUEUE_DEPTH",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "DEPTH": 0,
    "_ID": 261678143129
  },
  "type": "TX.QUEUE_DEPTH",
  "value": ""
}
```

Notes

The returned value in this call is still being worked on. 0 really means zero and 1 means more than zero. Note that it's possible for there to be no additional messages in the queue (ie, 0), however there could still be one currently transmitting. This API call is subject to change.

Endpoint: TX.GET_STATUS

Description

See if the radio is currently transmitting.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- TRANSMITTING - True or False
- MESSAGE - The varicode-encoded version of what's currently being sent.
- _ID - Unique transaction ID

Return Type

TX.STATUS

Example Sent Data

```
{  
  "params": {},  
  "type": "TX.GET_STATUS",  
  "value": ""  
}
```

Example Reply

```
{  
  "params": {  
    "MESSAGE": "",  
    "TRANSMITTING": false,  
    "_ID": 261678216291  
  },  
  "type": "TX.STATUS",  
  "value": ""  
}
```

Notes

None

Endpoint: TX.SEND_HB

Description

Send a heartbeat.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

TX.SEND_HB

Example Sent Data

```
{
  "params": {},
  "type": "TX.SEND_HB",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261678261604
  },
  "type": "TX.SEND_HB",
  "value": ""
}
```

Notes

If a grid square has been configured, the first four digits are automatically sent as part of the heartbeat.

Endpoint: TX.STOP_TX

Description

Immediately halt transmission.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

TX.STOP

Example Sent Data

```
{
  "params": {},
  "type": "TX.STOP_TX",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261678310759
  },
  "type": "TX.STOP",
  "value": ""
}
```

Notes

None

Endpoint: TX.SEND_CQ

Description

Send a CQ message as defined by the CQ message field in the setup.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

TX.SEND_CQ

Example Sent Data

```
{
  "params": {},
  "type": "TX.SEND_CQ",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261678369046
  },
  "type": "TX.SEND_CQ",
  "value": ""
}
```

Notes

None

Endpoint: TX.SEND_INFO

Description

Send the Info field.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

TX.SEND_INFO

Example Sent Data

```
{
  "params": {},
  "type": "TX.SEND_INFO",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261678402915
  },
  "type": "TX.SEND_INFO",
  "value": ""
}
```


Notes

None

Endpoint: TX.SEND_STATUS

Description

Send the Status field.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

TX.SEND_STATUS

Example Sent Data

```
{
  "params": {},
  "type": "TX.SEND_STATUS",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261678440401
  },
  "type": "TX.SEND_STATUS",
  "value": ""
}
```

Notes

None

Endpoint: TX.TX_ON

Description

Allow transmit.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

TX.TX

Example Sent Data

```
{
  "params": {},
  "type": "TX.TX_ON",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261678519206
  },
  "type": "TX.TX",
  "value": ""
}
```

Notes

None

Endpoint: TX.TX_OFF

Description

Disable transmit.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

TX.TX todo: duplicate

Example Sent Data

```
{
  "params": {},
  "type": "TX.TX_OFF",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261678493252
  },
  "type": "TX.TX",
  "value": ""
}
```

Notes

None

Endpoint: TX.GET_TEXT

Description

Return the contents of the lower center white pane.

Calling Value

None

Calling Params

None

Return Value

The contents of the lower center white pane (or False).

Return Params

- _ID - Unique transaction ID

Return Type

TX.TEXT

Example Sent Data

```
{
  "params": {},
  "type": "TX.GET_TEXT",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261678577171
  },
  "type": "TX.TEXT",
  "value": "WHO IS JOHN GALT?"
}
```

Notes

None

Endpoint: TX.CLEAR_TEXT

Description

Clear the contents of the center white pane.

Calling Value

None

Calling Params

None

Return Value

None

Return Params

- _ID - Unique transaction ID

Return Type

TX.TEXT

Example Sent Data

```
{
  "params": {},
  "type": "TX.CLEAR_TEXT",
  "value": ""
}
```

Example Reply

```
{
  "params": {
    "_ID": 261678604096
  },
  "type": "TX.TEXT",
  "value": ""
}
```

Notes

None

Endpoint: TX.SET_TEXT

Description

Put text into the center white pane, but don't send.

Calling Value

None

Calling Params

The text to be sent.

Return Value

None

Return Params

- SUCCESS - Did this succeed? (True or False)
- _ID - Unique transaction ID

Return Type

TX.TEXT

Example Sent Data

```
{
  "params": {},
  "type": "TX.SET_TEXT",
  "value": "WHO IS JOHN GALT?"
}
```

Example Reply

```
{
  "params": {
    "SUCCESS": true,
    "_ID": 261678660663
  },
  "type": "TX.TEXT",
```

```
"value": "WHO IS JOHN GALT?"
}
```

Notes

You cannot add or change text if there is text already queued. You can first call TX.GET_TEXT to see if it's possible to add text in the current state, or simply try to send and check the value of the returned SUCCESS parameter.

Endpoint: TX.SEND_MESSAGE

Description

Transmit this message.

Calling Value

None

Calling Params

The message to be transmitted.

Return Value

- ◦ SUCCESS - Did this succeed? True or False.

Return Params

- _ID - Unique transaction ID

Return Type

TX.MESSAGE

Example Sent Data

```
{
  "params": {},
  "type": "TX.SEND_MESSAGE",
  "value": "WHEREVER YOU GO, THERE YOU ARE"
}
```

Example Reply

```
{
  "params": {
```

```
    "SUCCESS": true,  
    "_ID": 261678871815  
  },  
  "type": "TX.MESSAGE",  
  "value": "NØGQ: WHEREVER YOU GO, THERE YOU ARE"  
}
```

Notes

You cannot add or change text if there is text already queued. You can first call TX.GET_TEXT to see if it's possible to add text in the current state, or simply try to send and check the value of the returned SUCCESS parameter.